

Version Control Systems

An Overview of  GitHub

Suresh Paul

Algorithm Basics and BW

What is Version Control?

- ▶ A system that **tracks changes** to documents/files  over time
- ▶ Allows you to:
 - ▶ See what changed, when, and by whom
 - ▶ Restore previous versions
 - ▶ Understand the history of a project
 - ▶ Recover from mistakes
- ▶ Essential for managing projects in any organization
- ▶ Supported file types:
 - ▶ `.md` (Markdown), `.txt`, `.csv`, `.html`, `.json`, `.xml`
 - ▶ Any plain text file

Why Version Control Matters in Business

Real-world scenarios:

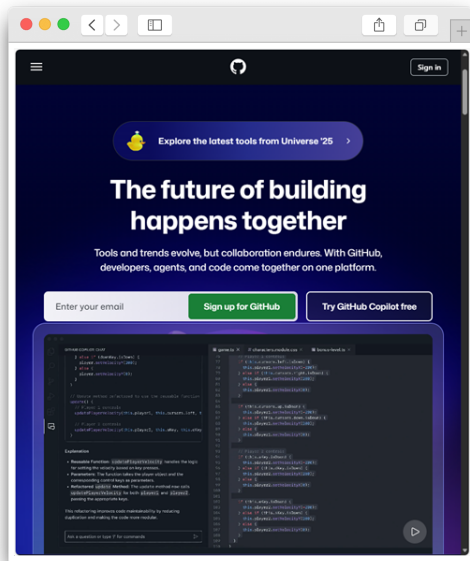
- ▶ Your manager asks: “What did our Q2 budget look like 3 weeks ago?”
- ▶ You accidentally delete critical data from a report
- ▶ Multiple team members need to track changes to project requirements
- ▶ Compliance/audit requires documentation of changes

Without version control:

- ▶ Multiple files like
 report_final.docx,  report_final2.docx,
 report_ACTUAL_final.docx

Today's Tool: GitHub

- ▶  **GitHub** = Web platform for version control
- ▶ Originally built for software developers
- ▶ Now used for project management, documentation, data, and *more*
- ▶ We'll use  GitHub's web interface (**no coding required!**)



Today's Hands-On Project

Scenario: You're creating a project status report for your team

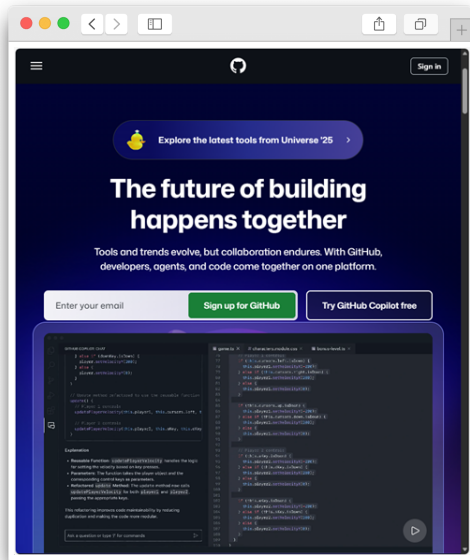
What we'll do:

1. Create a  **GitHub** account
2. Create a new repository (project folder)
3. Create a  **markdown** document
4. Make several updates to track project progress
5. View version history
6. Restore a previous version

Step 1: Create GitHub Account

Follow along on your computer:

1. Go to <https://github.com>
2. Click “Sign up”
3. Enter your email (university email)
4. Create a password
5. Choose a username
6. Verify your account via email
7. Select “Free” account when prompted

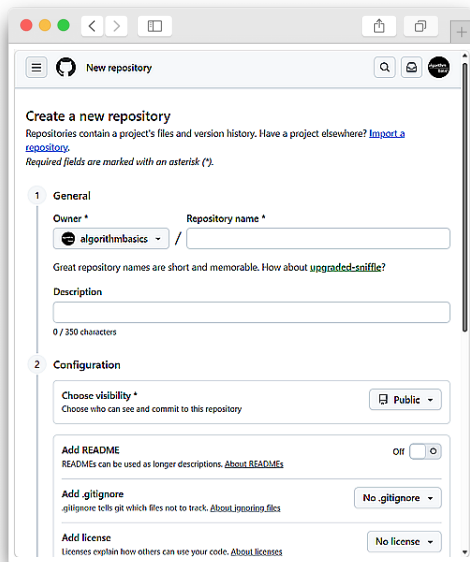


Step 2: Create a Repository

Repository = A project folder that tracks versions

1. Click the green “New” button
2. Repository name:
`project-status-report`
3. Description:
“Weekly project status tracking”
4. Select “Public”
(for learning purposes)
5. Check “Add a README file”
6. Click “Create repository”

You now have a project space with version control enabled!



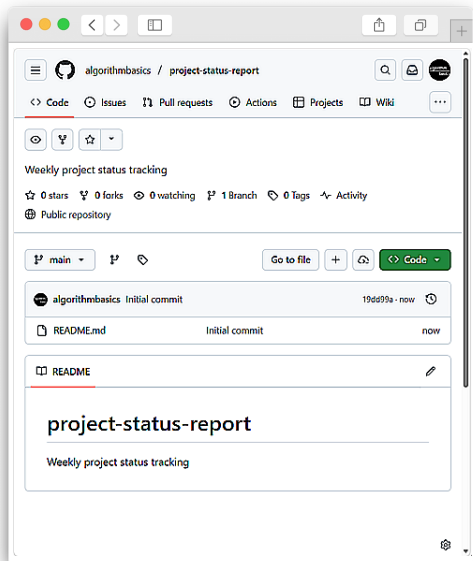
The screenshot shows the 'New repository' page on GitHub. The page title is 'New repository'. Below the title, it says 'Create a new repository' and 'Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository.](#) Required fields are marked with an asterisk (*).' The page is divided into two sections: '1 General' and '2 Configuration'. In the 'General' section, there is a form with 'Owner *' set to 'algorithmbasics' and 'Repository name *' as an empty text field. Below this, it says 'Great repository names are short and memorable. How about [upgraded-sniffle?](#)'. There is also a 'Description' text field with a character count '0 / 350 characters'. In the '2 Configuration' section, there is a 'Choose visibility *' dropdown set to 'Public'. Below that, there are three checkboxes: 'Add README' (checked), 'Add .gitignore' (checked), and 'Add license' (checked). Each checkbox has a link to 'About' followed by the respective name (READMEs, ignoring files, licenses).

Understanding What You Just Created

- ▶ You should see a file called `README.md`
- ▶ The `.md` extension = Markdown (simple formatting language)
- ▶ Markdown is like a simplified Word document
- ▶ Used widely in tech and project management

Key concept:

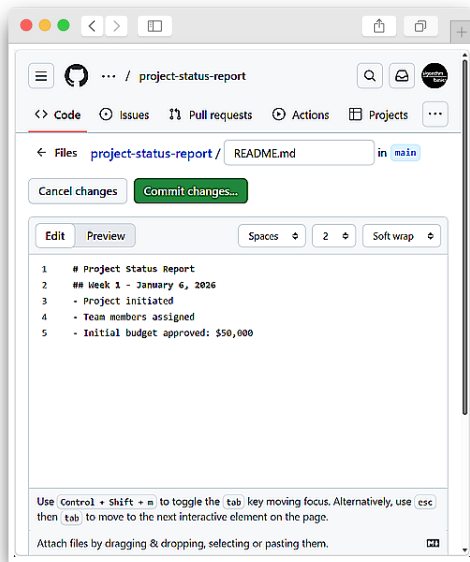
Every change you make will be tracked with a “commit” (a saved snapshot)



Step 3: Edit Your First Document

1. Click on README.md to open it
2. Click the pencil icon (Edit this file) on the right
3. Delete the default text
4. Type the following:

```
# Project Status Report
## Week 1 - January 6, 2026
- Project initiated
- Team members assigned
- Initial budget approved:
$50,000
```



Step 4: Your First Commit

Commit =  Saving a version with a description

Scroll to bottom of the page, you'll see  "Commit changes"

1. In the "Commit message" box, type: `Add Week 1 status`
2. Leave "Commit directly to main" selected
3. Click  "Commit changes"

Congratulations! You just created your first tracked version.

Best practice: Write clear commit messages that describe WHAT changed

Step 5: Make More Changes

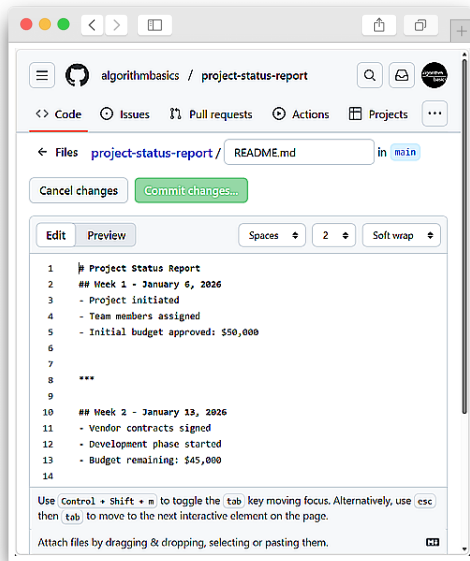
Let's simulate a real project with multiple updates:

Update 1: Add Week 2

1. Click README.md → Click pencil icon
2. Add after Week 1 content:

```
## Week 2 - January 13, 2026
- Vendor contracts signed
- Development phase started
- Budget remaining: $45,000
```

3. **Commit message:**
Add Week 2 status
4. Click  "Commit changes"



Step 6: Simulate a Mistake

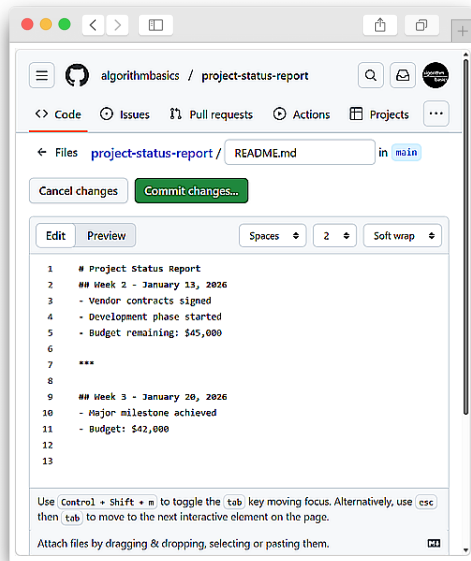
Let's simulate accidentally deleting important information:

1. Click README.md → Click pencil icon
2. Delete the entire Week 1 section
3. Add a new week instead:

```
## Week 3 - January 20, 2026
- Major milestone achieved
- Budget: $42,000
```

4. Commit message:
Add Week 3, remove old data
5. Click  "Commit changes"

We deleted Week 1 data.
Can we get it back?



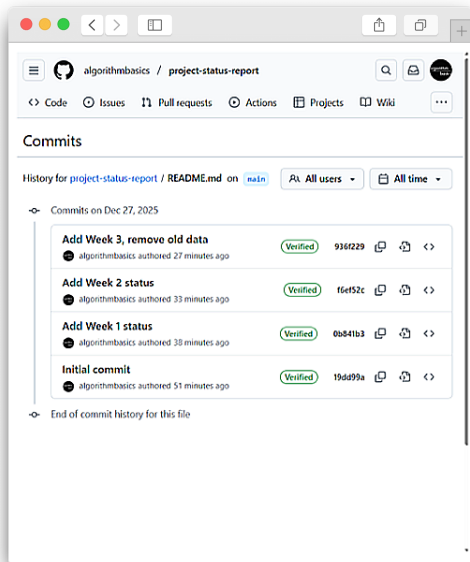
Step 7: View Version History

This is where version control is useful!

1. Click on `README.md`
(you should be on the main page)
2. Click “**History**” button
(top right, looks like a clock icon)
3. You'll see all your commits listed with:
 - ▶ Commit message, Date and time
 - ▶ Who made the change, Unique ID
4. Click on “Add Week 2 status” commit
5. You can see the file as it existed at that point!

Key insight:

Nothing is ever truly lost in version control



Step 8: Restore Previous Version

Let's recover our deleted Week 1 data:

1. Go back to History (click back button)
2. Find the commit "Add Week 2 status" (when Week 1 still existed)
3. Click on that commit
4. Click on README.md filename
5. You'll see the old version - click "Raw" button
6.  Copy all the content (Ctrl+A, Ctrl+C)
7. Go back to your main repository page
8. Click README.md → Edit (pencil icon)
9.  Paste the old content
10. Commit message: Restore Week 1 and Week 2 data
11. Click  "Commit changes"

Viewing Changes: The Diff View

Another powerful feature: See exactly what changed

1. Go to History again
2. Click on any commit
3. You'll see a "diff" (difference) view:
 - ▶ Red/minus sign = deleted content
 - ▶ Green/plus sign = added content
4. This makes it easy to track exactly what changed between versions

In business:

Essential for reviewing contract changes, budget updates, requirements modifications

Practice Exercise

On your own:

1. Add Week 4 and Week 5 status updates
2. Make at least 2 separate commits (one for each week)
3. Use meaningful commit messages
4. View the history
5. Look at the diff for your changes

Key Takeaways: GitHub Version Control

- ▶ **Repository** = Project folder with version tracking
- ▶ **Commit** = Saving a snapshot with a description
- ▶ **History** = Complete record of all changes
- ▶ **Diff** = Visual comparison between versions
- ▶ Every change is tracked: what, when, who
- ▶ You can always go back to previous versions
- ▶ Write clear commit messages!

Beyond the Basics

What we didn't cover today (but GitHub can do):

- ▶ Collaboration: Multiple people working on the same project
- ▶ Branches: Testing changes without affecting the main version
- ▶ Pull requests: Reviewing changes before merging
- ▶ Issues: Tracking tasks and bugs

These are important for team projects but not needed for basic version control!

Questions?

Questions and Discussion